

1. Introduction and Motivation

Neural networks are increasingly used as surrogate models in optimization when the relationship between decisions and outcomes is unknown, nonlinear, or difficult to model directly.

However, a neural network that predicts well is not necessarily easy to optimize over. When ReLU networks are embedded into mixed-integer optimization models, larger networks can introduce many additional variables and constraints.

This creates a trade-off between predictive accuracy and optimization tractability.

Key Question:

What neural network should we use when the goal is not only prediction, but downstream optimization?

2. Case Study and Background

We use a synthetic marketing allocation problem to study neural-network selection for downstream optimization.

The decision maker allocates a fixed advertising budget across multiple channels and chooses one social-media platform. The goal is to maximize total sales.

The true sales response is nonlinear. We generate data from this response function, train neural networks as surrogate models, and then optimize over the trained networks.

Component	Role in the Case Study
Continuous variables ($x \in \mathbb{R}^p$)	Budget allocated to each advertising channel
Binary variables ($z \in \{0,1\}^q$)	Special ordered set of type 1 / one-hot encoding of which social media platform is used
Context parameters ($\zeta \in \mathbb{R}^3$)	Fixed price, last-month sales, competitor spending
Predicted outcome ($\hat{y}$)	NN-predicted total sales, $\hat{y} = \text{NN}(x, z, \zeta)$
Goal	Maximize predicted outcome under a budget constraint

$$\begin{aligned} \max \quad & \hat{y} = \text{NN}(x, z, \zeta) \\ \text{s. t.} \quad & \sum_{i=1}^p x_i \leq B \\ & \sum_{j=1}^q z_j = 1 \\ & 0 \leq x \leq u, \quad z \in \{0,1\}^q \end{aligned}$$

B: total advertising budget
u: channel-specific spending limits
 ζ : fixed contextual variables

3. NN Family and Training

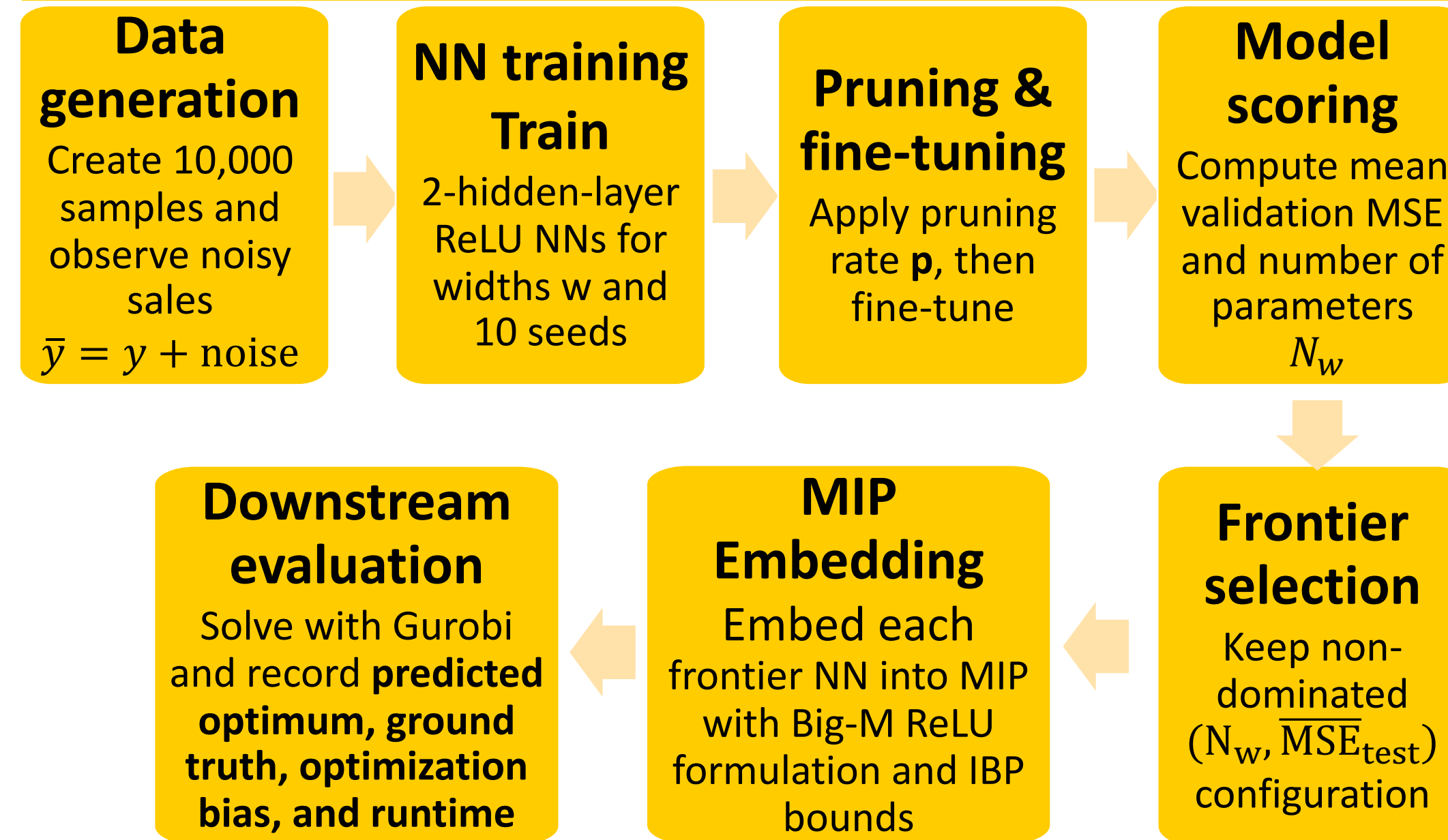
Architecture:

- 2-hidden-layer ReLU networks
- Input dimension $d = 14$
- 19 widths
 $w \in \{10, 20, \dots, 100, 200, \dots, 1000\}$
- 10 pruning rates, including unpruned
 $p \in \{0.0, 0.1, 0.2, \dots, 0.9\}$
- 10 random seeds per configuration
- 80:20 train/validation split

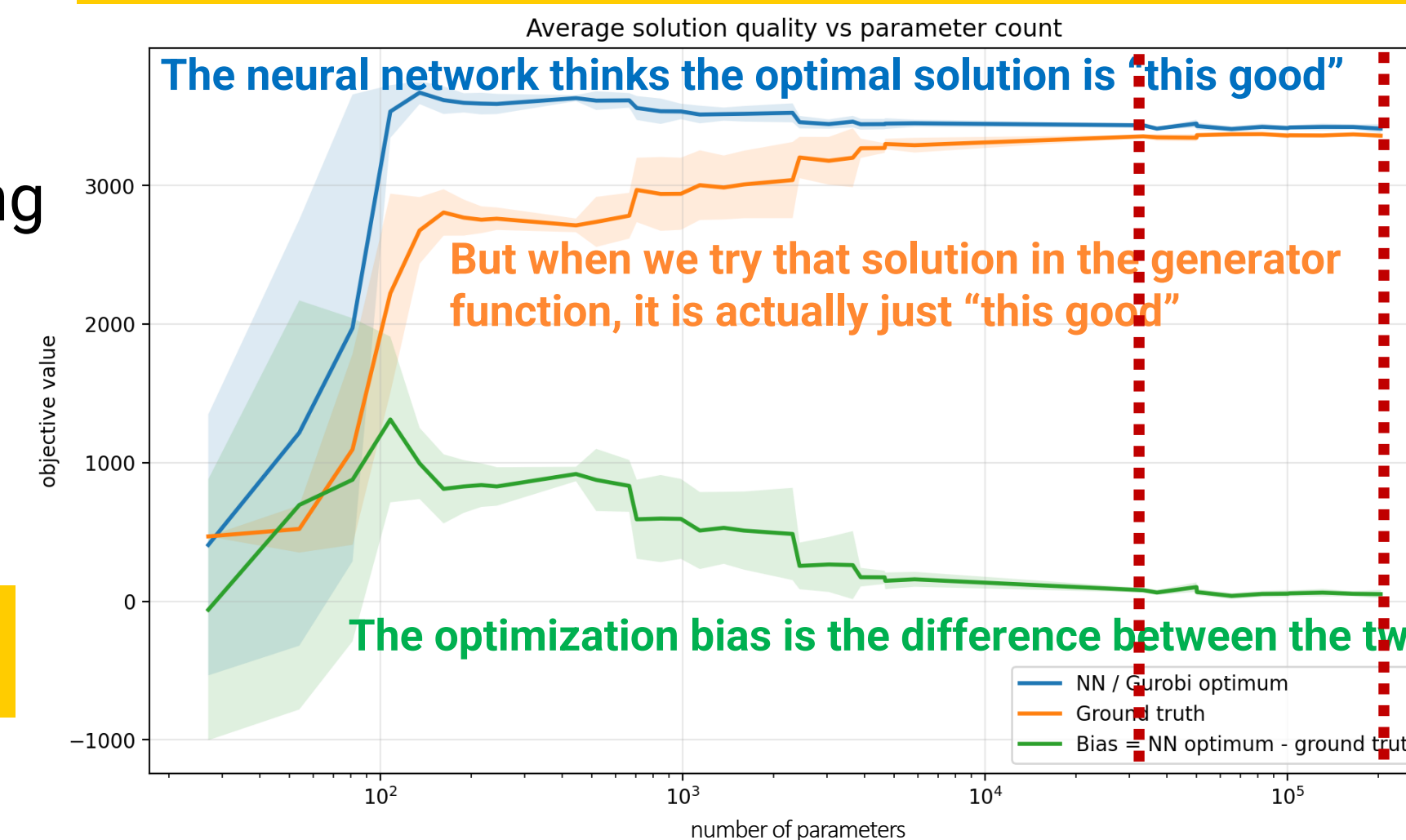
After pruning:

- Fine-tune each model
 - Compute validation MSE
 - Compute number of parameters
 $N_w(d, w, p) = (dw + w^2 + w)(1 - p)$
- This gives candidate neural networks with different prediction accuracy and optimization complexity.

4. Experimental Design



6. Downstream Results



Larger NNs improve ground-truth solution quality and reduce optimization bias.

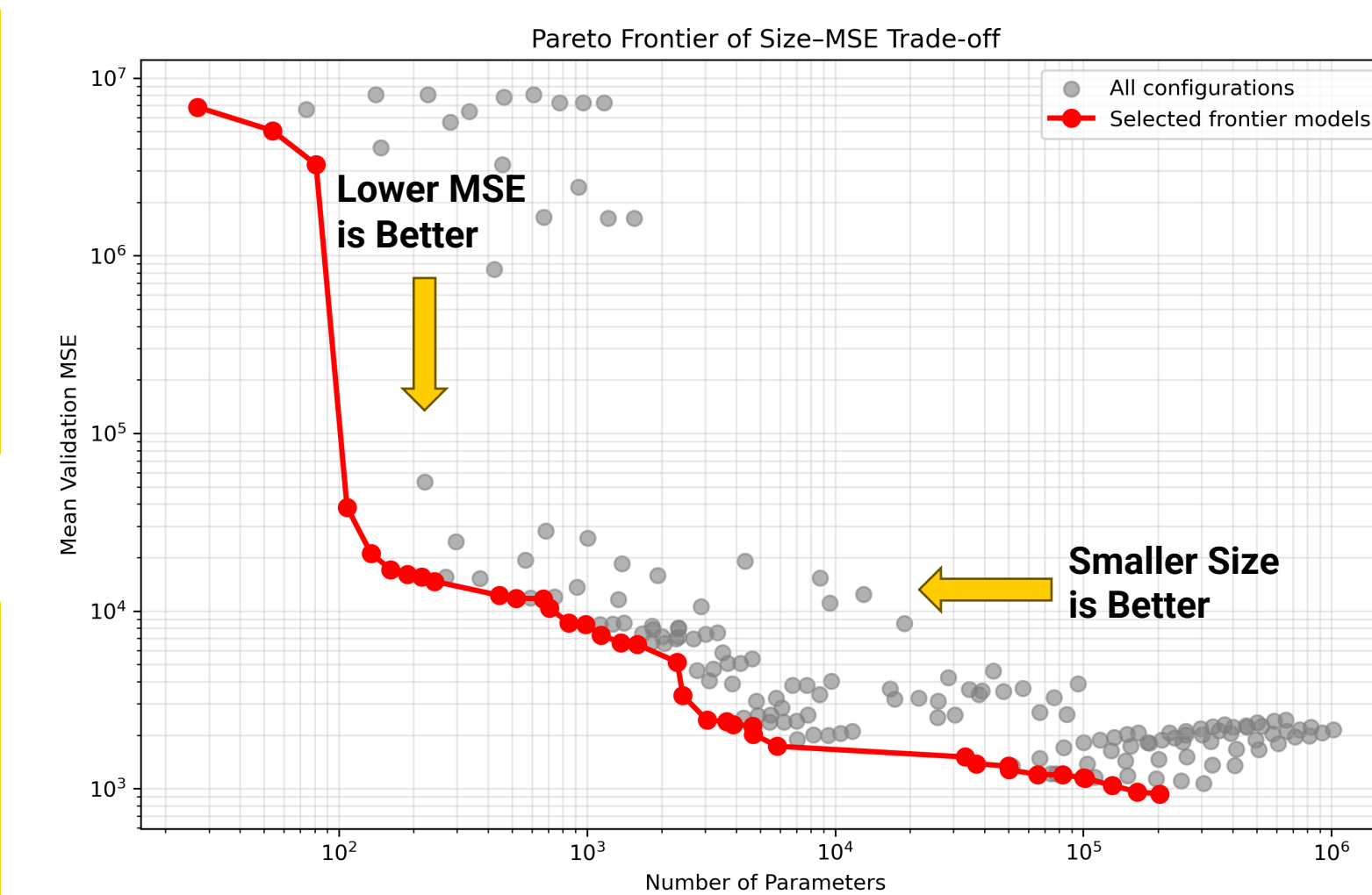
The dashed red lines distinguish models with 33,360 vs. 203,400 parameters. With more parameters, the gap between prediction and ground truth decreases from 2.37% to 1.52%, whereas the ground truth value increases by 0.14%. However, the average solve time increases from 2.01s to 270.13s (over 134× longer).

7. Key Takeaways

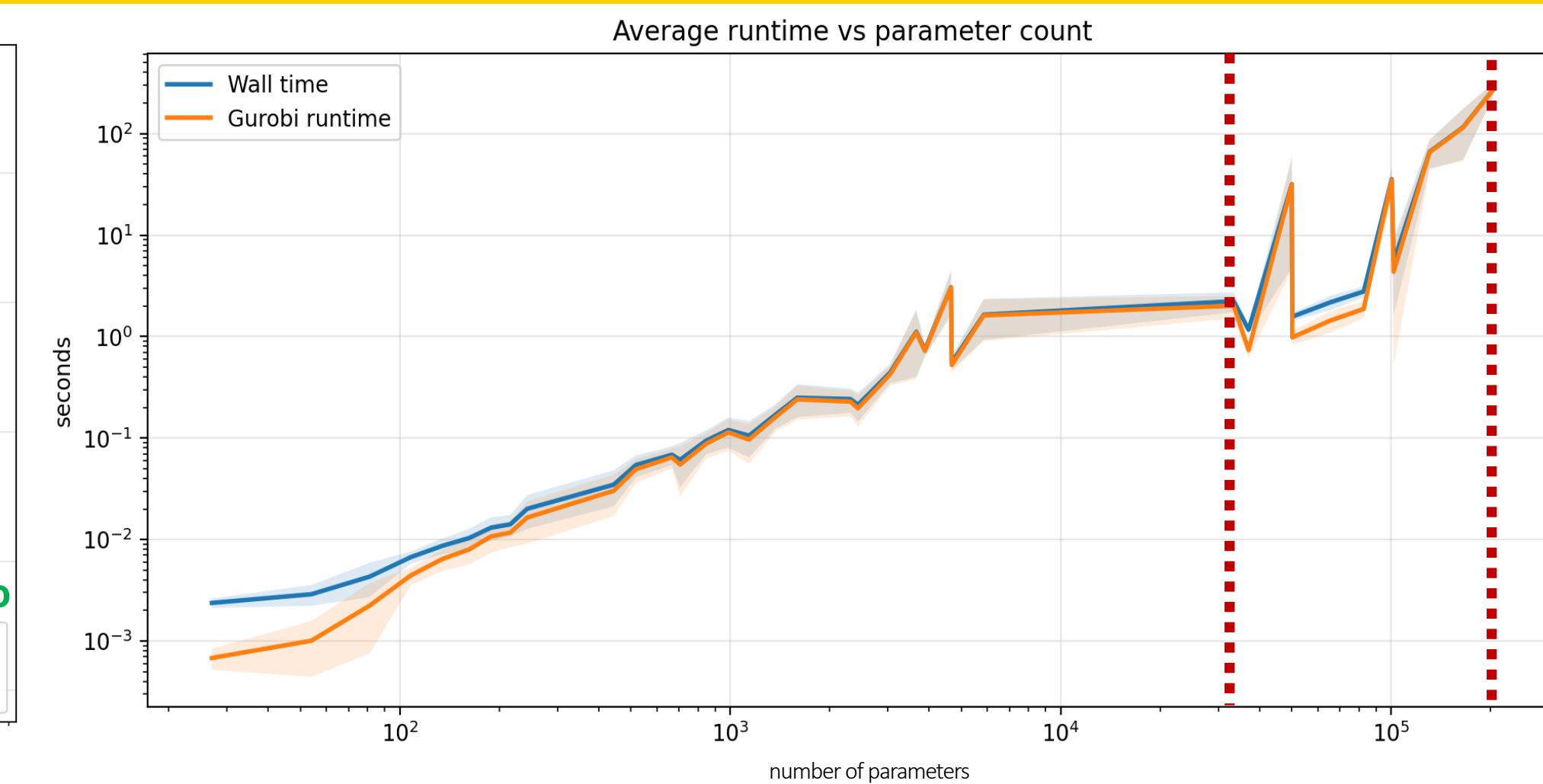
- Prediction accuracy alone is not enough for selecting a surrogate NN.
- Larger NNs can improve downstream solution quality and reduce optimization bias.
- Larger NNs also make the embedded MIP harder to solve.
- The best NN should balance predictive quality, decision quality, and tractability.

A slightly more accurate NN may be too large to optimize over efficiently.

5. Size-Accuracy Frontier



We keep non-dominated models that balance validation MSE and size.



However, MIP solve time increases sharply as the embedded NN becomes larger.

